Documentation de conception

Projet Génie Logiciel

Groupe 1, gl06

ZIVKOVIC Lucas, SULLET Benjamin, ZOUGGARI Samy,
TRAORÉ Fié Victor, HURSTEL Mathis

21 Janvier 2025

Grenoble INP - Ensimag
Génie Logiciel 2024-2025

Sommaire

Contents

1 Introduction **2**

2 Architecture **3**
2.1 Gestion de la Compilation 3
2.2 Gestion du Lexer 3
2.3 Gestion du Parser 4
2.4 Vérifications Contextuelles et Code Assembleur 4
2.4.1 Structure Générale des Classes 4
2.4.2 Gestion des Classes 4
2.4.3 Gestion des Champs (`DeclField.java`) 5
2.4.4 Gestion des Méthodes (`DeclMethod.java`) 5
2.4.5 Gestion du Main 5
2.4.6 Gestion des Expressions (`AbstractExpr.java`) 6
2.4.7 Conclusion 6

3 Conception de la génération de code **7**
3.1 Sous-langage Hello World 7
3.2 Deca sans objet 8
3.2.1 Le parcours de l'arbre décoré 8
3.2.2 Les messages d'erreurs 9
3.3 Deca essentiel 9
3.3.1 Premier parcours 9
3.3.2 Deuxième parcours 10

4 La gestion des registres **11**

5 Limites **11**

6 Conclusion **11**

1 Introduction

Ce documment a été rédigé dans le cadre de notre projet de Génie Logiciel de deuxième année d'école d'ingénieur visant à concevoir en Java un compilateur pour le langage Deca (petit langage ressemblant à Java). Ce projet qui s'est étalé sur une durée de 6 semaines est une pièce centrale dans la formation des ingénieurs ENSIMAG. Ces intérets sont nombreux comme :

- Nous permettre de découvrir le fonctionnement du travail en mode projet, en mettant en place une bonne organisation d'équipe et en nous initiant aux méthodes de développement agiles (TDD, Intégration continue,...) qui nous seront probablement utiles dans notre vie professionnelle.
- Etre en capacité de répondre le plus fidèlement possible à un cahier des charge exigeant dans un délais assez court.
- Mieux comprendre les languages de programmation grâce au fait d'avoir pu se pencher en profondeur sur les différents aspects de la compilation et ainsi progresser et programmation de manière générale.
- Nous sensibiliser à la validation par les tests notament, afin d'avoir un produit qui se rapproche le plus possible d'un compilateur "zero défaut", car produire des outils de qualité est essentiel pour tout ingénieur.
- Utiliser des outils d'aide au développement comme Maven, Git, un bon IDE...

nous a permis de découvrir ce qu'est réellement que le travail en mode projet, se rapprochant de ce que nous pourront être amené à faire dans notre vie professionnelle futur. En effet, durant cette période nous avons pu expérimenté des méthodes de développement agiles, ainsi que la nécessité de suivre un cahier des charges et de gérer l'importante quantité de travail à fournir dans le temps imparti. Toutefois les buts de ce projet n'était pas seulement en matière de génie Logiciel, mais aussi en terme de programmation. Même si seulement peu d'entre nous seront peut-être amené à devoir reproduire un compilateur à l'avenir, ce projet nous permet quand même de nous pencher en profondeur sur les différentes étapes de la compilation (analyse lexicale, syntaxique et génération de code) ce qui est très utile pour mieux comprendre les langages de programmation de manière générale. Enfin, une part importante du projet a été la validation notament par la réalisations d'un grand nombre de tests afin de se rapprocher le plus possible d'un produit "zero défaut" afin de nous sensibilser à l'importance de la qualité.

Cette documentation peut-être vue comme un approfondissement du Guide Utilisateur puisqu'il a pour but de fournir une vision plus détaillée des différentes parties de notre compilateur en m'étant l'accent sur nos choix d'implémentation, les structures de données utilisées ainsi que les différents algorithmes.

A l'issue de la lecture de ce document vous aurez une compréhension plus claire de la conception architecturale de notre produit. Les différentes parties abordées sont listées ci-dessous :

- Une présentation générale du compilateur dans son ensemble, présentant les différentes étapes de la compilation.
- Une partie sur la conception de l'analyse contextuelle dans laquelle on retrouvera ses objectifs mais aussi des détails techniques quant à notre implémentation.
- Une partie sur la conception de la génération de code présentant ses objectifs, les différentes étpaes de la génération ainsi que les choix architecturaux que nous avons fait.

2 Architecture

Pour réaliser ce compilateur Deca, nous avons adopté l'architecture suivante, permettant de gérer efficacement la compilation des programmes écrits dans ce langage. Cette architecture s'appuie sur plusieurs fichiers et classes Java, ainsi que sur des outils spécifiques pour le traitement lexical, syntaxique et contextuel, jusqu'à la génération du code assembleur.

2.1 Gestion de la Compilation

Pour gérer la compilation des programmes Deca, nous utilisons la classe `DecacMain.java`. Elle sert de point d'entrée pour compiler le code en effectuant toutes les vérifications nécessaires à une compilation correcte. Ces vérifications incluent :

- Les vérifications lexicales ;
- Les vérifications syntaxiques ;
- Les vérifications contextuelles ;
- Enfin, les vérifications du code assembleur.

Ces vérifications sont implémentées dans la classe `DecacCompiler.java`, qui assure la compilation complète des programmes écrits en Deca. Par ailleurs, la classe `CompilerOptions.java` gère toutes les options applicables à la commande `decac`. Cette commande, à son tour, lance la classe `DecacMain.java`. Ainsi, pour ajouter ou modifier une option, il suffit de mettre à jour la classe `CompilerOptions.java`.

2.2 Gestion du Lexer

Le lexer est défini dans le fichier `DecaLexer.g4`, situé dans le répertoire `src/main/antlr4/fr/ensimag/deca/syntax`. Ce fichier spécifie l'ensemble des mots reconnus par le langage Deca, comme par exemple :

```
EQUALS : '=';
```

Pour la maintenance, si vous souhaitez ajouter un nouveau mot au langage, il suffit de le déclarer dans ce fichier.

2.3 Gestion du Parser

Les vérifications syntaxiques sont effectuées par le parser, défini dans le fichier `DecaParser.g4`. Ce fichier contient toutes les règles de la grammaire du langage Deca, telles qu'elles ont été spécifiées au début du projet. Toute modification ou ajout à la grammaire doit être réalisée dans ce fichier, en veillant à insérer les nouvelles règles au bon endroit pour ne pas perturber le reste de la grammaire.

Le parser permet la création des classes nécessaires aux vérifications syntaxiques contextuelles et à la génération du code assembleur. La création des arbres syntaxiques se fait de manière ascendante, les résultats étant progressivement remontés pour aboutir à une structure exploitable pour les étapes suivantes.

2.4 Vérifications Contextuelles et Code Assembleur

L'architecture des classes gère à la fois les vérifications syntaxiques contextuelles et les vérifications effectuées sur le code assembleur généré. Ces étapes garantissent la validité du code produit, en assurant la cohérence contextuelle et la correction lors de l'exécution en assembleur.

2.4.1 Structure Générale des Classes

Toutes les classes utilisées dans cette étape héritent soit de `Tree`, soit de `ListTree`. Le fichier `DecacMain.java` constitue le point d'entrée de la compilation en appelant la classe `Program.java`, qui hérite de `AbstractProgram.java`. Cette dernière est essentielle pour gérer les déclarations de classes ainsi que le point d'entrée `main`, deux éléments indispensables pour compiler les programmes en langage Deca.

2.4.2 Gestion des Classes

La gestion des classes repose sur l'architecture suivante :

- La classe `ListDeclClass.java` gère la liste des déclarations de classes.
- Chaque classe déclarée est représentée par `DeclClass.java`, qui inclut :
  - Le nom de la classe ;
  - Le nom de la classe parente (`extends`), si applicable ;
  - Ces noms sont des instances de la classe `Identifier.java`.
- La classe `DeclClass.java` contient également un corps (`body`) représenté par une liste d'éléments dérivés de `AbstractBody`. Ces éléments peuvent être :

– Des déclarations de champs (`DeclField.java`) ;
– Des déclarations de méthodes (`DeclMethod.java`).

2.4.3 Gestion des Champs (`DeclField.java`)

La classe `DeclField.java` gère les champs d'une classe. Les informations suivantes y sont définies :

- La visibilité du champ (`protected` ou `public`) ;
- Le type du champ ;
- Le nom du champ ;
- Une éventuelle initialisation.

Pour ajouter une nouvelle visibilité au langage Deca, il faut :

- Modifier la classe `Visibility.java` pour intégrer la nouvelle option ;
- Apporter les ajustements nécessaires dans `DeclField.java`.

2.4.4 Gestion des Méthodes (`DeclMethod.java`)

La gestion des méthodes repose sur la classe `DeclMethod.java`, qui spécifie :

- Le type de retour de la méthode ;
- Le nom de la méthode ;
- Les paramètres de la méthode ;
- Le corps de la méthode.

Le corps d'une méthode est similaire à celui de la méthode `main`, avec une extension pour gérer le code assembleur. Pour ajouter une nouvelle visibilité aux méthodes, il est nécessaire de modifier `DeclMethod.java` ainsi que les classes associées à la gestion de la visibilité.

2.4.5 Gestion du Main

La méthode `main` est décomposée en deux parties principales :

1. **Liste de déclarations de variables** : La classe `ListDeclVar.java` gère cette liste. Une déclaration de variable inclut :
   - Le type de la variable ;
   - Le nom de la variable ;
   - Une éventuelle initialisation.

Les déclarations de variables sont similaires à celles des champs (`DeclField.java`), mais sans notion de visibilité.

2. **Liste des instructions** : Les instructions sont représentées par des instances de `AbstractInst.java`. Cette classe englobe plusieurs types d'instructions, notamment :

   - `While.java`, qui contient une condition (`AbstractExpr`) et un corps (liste d'instructions) ;
   - `NoOperation.java`, qui ne réalise aucune opération ;
   - `IfThenElse.java`, qui inclut une condition (`AbstractExpr`) et deux listes d'instructions pour les blocs `then` et `else` ;
   - `Return.java`, qui contient une expression de retour ;
   - `AbstractPrint.java`, qui gère toutes les variantes d'affichage (`print`, `println`, etc.) et prend en argument une expression (`AbstractExpr`).

   Pour ajouter une nouvelle instruction, il suffit de créer une classe qui hérite de `AbstractInst.java`.

2.4.6 Gestion des Expressions (`AbstractExpr.java`)

Les expressions (`AbstractExpr.java`) représentent l'ensemble des opérations et des valeurs permises dans le langage Deca :

- `AbstractUnaryExpr`, pour les opérations unaires (`-`, `!`) sur un opérande (`AbstractExpr`) ;
- `AbstractReadExpr`, qui permet de lire des `int` ou des `float` ;
- `AbstractBinaryExpr`, pour les opérations binaires (`=`, `==`, `!=`, `||`, `&&`, `+`, `-`, `*`, `/`, `%`, `<`, `>`, `<=`, `>=`) sur deux opérandes (`AbstractExpr`) ;
- Les littéraux (`LiteralExpr`), représentant des constantes comme `int`, `float`, `String`, `true`, `false`, `this`, ou `null`.

Pour étendre les expressions du langage Deca, il faut ajouter une classe dérivée de `AbstractExpr.java` correspondant à la nouvelle opération ou valeur.

2.4.7 Conclusion

Cette architecture modulaire permet une gestion claire et flexible des vérifications contextuelles et de l'exécution en assembleur. Les extensions du langage Deca, qu'il s'agisse d'ajouter des instructions, des visibilités ou des opérations, peuvent être réalisées en modifiant uniquement les composants concernés.

3 Conception de la génération de code

Une fois les étapes d'analyse lexicale, syntaxique et contextuelles terminées, nous voici arrivé à la dernière étape de la compilation dans le cadre de notre projet: La génération de code. L'objectif est de produire une suite d'intsructions écrites en langage d'assemblage qui pourront ensuite être exécutées par la machine abstraite qui nous a été fournie (`ima`). Pour cette partie là du projet spécialement nous avons adopté une approche incrémentale. Ceci nous semblait être un bon moyen d'avancer de manière sûr et de sorte que l'on ait régulièrement de nouvelles fonctionnalités du langage Deca qui soient prises en compte petit à petit. Bien évidement nous avons mis en place des tests de non régressions afin de s'assurer qu'à chaque incrément nous n'aillons pas "casser" ce qui fonctionnait précédemment (plus de details à ce sujet dans la documentation de validation). Les différents incréments ont été choisis en premier lieu en fonction des différents sous-langages de Deca qui existent : Hello World dans lequel on peut uniquement afficher des chaînes de caractères à l'aide de l'instruction `println("Ma chaîne de caractère")`, Deca sans objet dans lequel on peut effectuer des opérations arithmétiques et booléennes ainsi qu'utiliser des structures de contrôles (`if` et `while`), et enfin le Deca essentiel qui ajoute la possibilité de créer des classes et des méthodes en plus des fonctionnalités des deux autres sous-langages. Toutefois pour les derniers sous-langages mentionnés nous avons décidé de réduire la taille des incréments en ne gérant par exemple pas toutes les opérations aritjmétiques et les structures de contrôle d'un seul coup. En effet ce choix fût motivé par le fait qu'il serait plus simple de paralléliser notre travail et ainsi d'être plus efficace. Les prochaines sous-sections vont détaillées la conception des différent incréments que nous venons de mentionner.

3.1 Sous-langage Hello World

Après avoir regardé les différentes vidéos et s'être bien approprié le sujet du projet, notre premier objectif a été de réussir à compiler un fichier du sous-langage Hello World. Pour ce premier incréments la majorité du travail concernait les étapes d'analyse lexicale et syntaxique étant donné que les méhodes permettant la génération de codes étaient dans ce cas là déjà implémentées.

```
6
7 {
8         println("Bonjour");
9
10 }
```

Figure 1: Exemple d'un fichier Deca accepté dans le sous-langage Hello world

3.2 Deca sans objet

Nous nous sommes par conséquent rapidement penché sur ce qui était notre deuxième incrément : Deca sans objet. Ici il nous a fallut écrire des méthodes afin de pouvoir générer le code assembleur exécutable par la suite à l'aide de la commande `ima monfichier.ass`.

Comme expliqué précédemment, les étapes d'analyse lexicale, syntaxique et contextuelles on permis de générer un arbre abstrait contenant les différents token du fichier source .deca. De plus à la fin de l'étape B cet arbre est décoré par des attributs de la grammaire abstraite qui sont mémorisés afin de faciliter la génération de code.

3.2.1 Le parcours de l'arbre décoré

L'algorithme utilisé pour la génération de code est en fait un parcours d'arbre. Il consiste en effet à parcourir les différents noeuds de l'arbre abstrait décoré, et à chaque noeud appeler une méthode correspondante, qui va elle même appeler une autre méthode de manière récursive afin de descendre dans l'arbre jusqu'à arriver au niveau d'une feuille. Ensuite, les instructions assembleur à écrire dans le fichier .ass sont ajoutées et stockés dans l'attribut `program` de la classe DecacCompiler, dans une liste chaînée.

La génération de code suit la grammaire d'arbre et commence donc par l'appelle de la méthode `codeGenProgram` définie dans la classe AbstractProgram.java. Ensuite, chaque noeud est décrit par une classe java dans laquelle on retrouve différentes méthodes utiles à la génération de code. Dans un premier temps nous gérons la déclaration des variables qui se fait par un appel à la méthode `codeGenDeclVar`. Cette méthode va récupérer les informations utiles qui sont contenues dans l'arbre décoré (nom, type, valeur...) et va stocker cette variable sur le haut de la pile. Une fois les variables déclarées, viennent les instructions (opérations arithmétiques, booléennes, structure de contrôle...).

Nous avons décidé de faire des méthodes différentes pour chaque type d'utilisation du noeud. Ainsi par exemple dans la classe Multiply.java on retrouve les méthodes principales suivantes :

- `evalueExpr` : qui va s'appeler récursivement jusqu'à stocker la valeur des deux opérandes dans des registres distinct puis executer la multiplication grâce à l'instruction assembleur `MUL` prévue à cet effet.
- `codeGenInst` : qui va être appelée lorsque l'opération arithmétique n'est pas stocké dans une variable.
- `codeGenPrint` : qui va faire sensiblement la même chose que `evalueExpr` mais qui va en plus afficher le résultat. Cette méthode sera donc appelée dans le instruction nécessitant un affichage (`println()`).

Ces trois types de méthodes sont présents dans tous les fichiers représentants des noeuds. Ce choix de séparer les utilisations des méthodes nous a permis d'avoir des méthodes pas trop longues et donc un code plus clair facilitant encore une fois le débogage et la compréhension globale.

3.2.2 Les messages d'erreurs

Nous avons voulu que notre code assembleur soit le plus "propre" possible afin d'en faciliter la lecture et la compréhension lors des phases de débogage, ainsi nous avons décider de placer toutes les étiquettes avec des messages d'erreurs à la fin du fichier .ass, juste avant les déclarations des méthodes. Les différents types d'erreurs qui peuvent être levés par `ima` à l'exécution sont :

- l'erreur de lecture : valeur entrée par l'utilisateur pas du type attendue;
- la division par 0, pour les entier ou les flottants;
- le débordement arithmétique sur les flottant;
- l'abscence de return lors de l'exécution d'une méthode;
- le déréférencement d'un objet null;
- le débordement de la pile ou du tas;

Ainsi, à part le premier point qui est géré tout seul par ima, pour tous les autres nous créons une étiquettes qui sera ajouter à la fin du programme à laquelle on sautera si l'erreur se produit.

3.3 Deca essentiel

La dernière étape du projet a été d'augmenter le nombre de programme Deca acceptable en faisant en sorte que notre compilateur puisse compiler des classes et des méthodes, et générer le code en conséquence. Pour ce qui est du programme principal, notre implémentation pour la génération de code ne change pas vraiment du cas Deca sans objet (à part les appel de méthode que nous aborderons par la suite). Ce qui change c'est que nous avons maintenant des classes que nous allons gérer avant toute choses. La génération de code pour un programme Deca objet va nécessiter d'effectuer deux parcours de l'arbre abstrait.

3.3.1 Premier parcours

Durant le premier parcours, la tâche principale va être de définir la table des méthodes, qui sera stockée dans la pile. Cependant, avant de créer cette table des méthodes nous avons décider de d'abord créer pour chaque instance d'une classe (caractérisé par une instance de DeclClass.java) une `ArrayList` de Label. Ces labels correspondent au noms des différentes méthodes d'une classe, et se verront dans le code assembleur sous la forme d'étiquettes. Ainsi, on stocke pour chaque classe toutes ces méthodes dans ce tableau. De plus, le concept d'abstraction permet à une classe de bénéficier des méthodes de sa classe mère, c'est pourquoi on ajoute dans le tableau de label de chaque classe les méthodes de sa super classe. Une fois ce tableau crée pour chaque classe du fichier source, on n'a plus qu'à le parcourir et remplir la pile avec les étiquettes des méthodes

avec les instructions assembleur prévues à cet effet (`new STRORE(register, offSet(GB))`. Un autre point important est le fait que l'offset d'une méthode par rapport au début de la table des méthodes de la classe à laquelle il appartient est une valeur qui est stockée dans chaque instance d'une méthode (représenté par la classe `MethodDefinition.java`, ceci sera très utile lors de l'appel de méthode pour retrouver où est déclaré chaque méthode.

3.3.2 Deuxième parcours

L'objectif du deuxième parcours était de générer les instructions assembleur permettant le codage de chaque méthode mais aussi des champs des différentes classes, et enfin de générer le code pour le programme principal. Dans l'optique de produire du code propre et respecter la structure imposée du code `.ass`, les instructions d'initialisation de champ et les corps des méthodes des classes ont été ajoutés en fin de fichier. Pour entrer dans le vif, la passe 2 nécessite de repenser les méthodes de nos classes JAVA.

En effet :

- Il entrait en jeu de nouveaux types de données : les paramètres et les Field des méthodes qu'il fallait gérer différemment ; ces derniers ont des adresses relatives, alors que jusque-là, nous traitions des variables avec adresses globales.
- Il y avait ensuite les instructions des initialisations et méthodes de classe qu'il fallait ajouter en fin du fichier `.ass`.
- Puis pour finir, il fallait refaire encore un tour de quelques méthodes JAVA pour intégrer les manipulations de tableau.

Ainsi, pour la génération des instructions d'initialisation, nous avons créé à cet effet deux classes dans `DeclClass.java`. La première méthode se charge de parcourir la classe pour générer les codes d'initialisations de ses champs en s'inspirant des exemples donnés dans le polycopié du projet.

La deuxième méthode parcourt la classe pour générer les instructions pour les méthodes de la Classe. Nous faisons cela pour chacune des classes du programme DECA.

Pour la génération des instructions des méthodes, nous avons ajouté à l'arborescence de Classe du projet (les fichiers du dossier `src/main/fr/ensimag/deca/tree`) deux nouvelles méthodes : `evalueExprEnd` et `codeGenInsMethod`. Le premier est une réplique de la méthode `evalueExpr` déjà présente dans nos classes JAVA pour le DECA sans Objet. Sauf que là, le `evalueExprEnd` ajoute les instructions en fin du fichier `.ass` et sait évaluer des Fields et des Paramètres.

La deuxième fonction, `codeGenInstMethod` ressemble beaucoup aussi à `codeGenInst` de DECA sans Objet, sauf qu'en plus, elle ajoute les instructions en fin du fichier `.ass` et sait également comment traiter des paramètres et des Field.

Après avoir géré ces déclarations de classes, la génération de code pour le programme principal reste pareille sauf qu'il fallait en plus traiter l'instruction `new` et les appels de méthodes ou les accès à des champs de classes. À cet effet, nous avons ajouté dans l'arborescence une nouvelle Classe `Dot.java` qui permet de gérer à la fois les Field et les Champs dans des définitions de classes et dans le programme principal et également les appels de méthodes.

4 La gestion des registres

Lors de la génération de code, il a été important de pouvoir gérer à notre convenance les zones mémoires dont nous disposions (la pile, le tas, les registres...). Nous avons décidé de créer une classe `GestionRegistre.java` dans laquelle nous allions effectuer les traitements nécessaires relatifs aux registres afin de ne pas poluer les méthodes du parcours d'arbre et de simplement avoir à appeler les méthodes de cette nouvelle classe. Pour ce qui est de la pile, des attributs entier nous ont permis de positionner les différents pointeurs (SP, LB et GB) et de les récupérer ou de les incrémenter au besoin.
De plus nous disposions de 16 registres R0...R15. Nous avons décidé de leur ajouter un attribut booléen afin de savoir si un registre est déjà utilisé ou non. Ceci nous avons pu créer une méthode qui nous permet de récupérer un registre libre pour pouvoir y stocker ce dont nous avions besoin. Enfin, cette classe nous permet aussi de gérer le cas où tous les registres sont occupé et que nous devons quand même stocker des données. Pour cela une méthode peut-être appelé pour mettre temporairement des données dans la pile afin de libérer des registres et poursuivre la compilation.

5 Limites

Comme première Limite, pour la GenCode, nous aurions avec un peu plus de temps pu refactoriser notre code pour ne pas avoir à écrire les méthodes evalueExprEnd et codegenInstMethod; calà n'a pas été anticipé dès le début du projet; nous l'assumons car nous avons tenu à être être optimal sur deca sans Objet comme conseillé par les Professeurs.

Ensuite, pour la génération de code des instruction IfthenElse, nous aurions pû également beaucoup factoriser le code car plusieurs boût de code se repète; mais une fois de plus, cela à été remarqué quand il n'y avait plus assez de temps pour risquer de tout casser.

6 Conclusion

Nous espérons que ce document vous aura permis de comprendre de manière détaillée comment s'architecturait notre compilateur et comment l'améliorer si il fallait implémenter de fonctionnalité du language Deca.